

ROXSTAR ASSESSMENT TEST

Spin Wheel Game System Requirements Document

Time Allocation: 2 days

Total Points: 100 points

1. Overview

Design and implement a real-time multiplayer spin wheel game system. Users can create spin wheels, join them by paying an entry fee, and compete for prize pools. The system must handle concurrent users, ensure fair coin distribution, and provide real-time updates to all participants.

2. Core Functionality Requirements

2.1 Spin Wheel Lifecycle (40 Points)

Initialize Spin Wheel (10 points)

- Only admins can create a spin wheel
- Only ONE active spin wheel at a time

Join Spin Wheel (10 points)

- Users pay entry fee in coins to join

Start Spin Wheel (10 points)

- Auto-start after 3 minutes OR manual start by admin
- Minimum participants required: 3 users
- If < 3 participants after 3 minutes: auto-abort and refund
- Generate random elimination sequence

Process Eliminations (10 points)

- Eliminate one user every 7 seconds

- Last remaining user wins
 - Distribute coins to winner and admin
-

2.2 Coin Distribution System (30 Points)

Entry Fee Distribution (15 points)

- When user joins, distribute entry fee according to configuration:
 - Winner Pool: X%
 - Admin Pool: Y%
 - App Pool: Z%
- Configuration should be database-driven and adjustable
- Track cumulative amounts in separate columns

Final Payout (15 points)

- Credit winner with accumulated winner pool
 - Credit admin with accumulated owner pool
 - Record all transactions in database
 - Ensure atomic coin operations (no partial credits/debits)
 - Handle concurrent coin updates safely
-

2.3 Real-Time Communication (30 Points)

3. Deliverables

Mandatory

1. Source Code: Complete, runnable implementation on a Git Repository
2. README.md: Setup instructions and how to run
3. Database Schema: SQL migration scripts or schema definition
4. Configuration: Environment variables and config files

Recommended

5. Architecture Diagram: High-level system design
 6. Test Cases: At least critical path coverage
 7. Edge Case Documentation: List of edge cases you handled
 8. Performance Considerations: Document your optimization decisions
-

4. Evaluation Criteria

Code Quality (Implicit in point distribution)

- Clean, readable, maintainable code
- Proper error handling and logging
- Appropriate use of design patterns

Problem Solving

- Identification of edge cases
 - Creative solutions to complex problems
-

5. Submission Guidelines

1. Create a Git repository (GitHub)
 2. Commit regularly with meaningful messages
 3. Include comprehensive README
 4. Be prepared to:
 - Walk through your code in a review session
 - Explain architectural decisions
 - Discuss alternative approaches considered
 - Defend your edge case handling
-

6. Questions & Clarifications

During the implementation, you may make reasonable assumptions. Document all assumptions in your README. If critical information is missing that would significantly impact your design, note it and proceed with what you consider the best approach.

Focus on: Correctness, scalability, and production-readiness over feature completeness.

All the Best 😊!!